Movie Recommendation System Report

1. Introduction

Movie recommendation systems play a crucial role in helping users discover films they are likely to enjoy from large catalogs. With streaming platforms like Netflix and Amazon Prime offering thousands of options, intelligent recommendation engines significantly enhance user experience by surfacing relevant content. These systems use user behavior, item characteristics, and advanced algorithms to personalize suggestions.

This project explores three widely used recommendation techniques: user-based collaborative filtering, item-based collaborative filtering, and a graph-based Pixie-inspired random walk algorithm. Each approach uses a different method to understand preferences and uncover connections between users and movies, ultimately producing personalized movie suggestions.

---

2. Dataset Description

The project uses the **MovieLens 100K** dataset, a well-known benchmark in the recommendation domain. It consists of:

- **943 users**
- **1,682 movies**
- **100,000 ratings**, ranging from 1 to 5 stars

The dataset is divided into three main files:

- u.data (user ratings)
- u.item (movie information)
- u.user (user demographic data)

For this project, the main features used were **user ID**, **movie ID**, and **ratings**. The timestamp column in the ratings was converted to readable dates. Additionally, missing values (such as a release date in one movie) were handled, and movie titles marked as "unknown" were noted.

The datasets were merged appropriately, and a user-movie rating matrix was constructed for collaborative filtering techniques. All cleaned datasets were saved into CSV format.

---

3. Methodology

This system includes the implementation of three recommendation methods:

User-Based Collaborative Filtering:
This approach identifies users who have similar rating patterns to the target user. Cosine similarity is used to compute pairwise similarities between users. Once similar users are found, the system recommends movies that those similar users liked but the target user hasn't watched yet.

Item-Based Collaborative Filtering:
In contrast to user-based methods, this approach focuses on finding movies that are similar to a given movie. It looks at how users rate pairs of movies and determines similarity based on these co-ratings using cosine similarity. When a user likes a movie, they are recommended similar movies that other users also liked.

Graph-Based Pixie Algorithm (Random Walk):
This approach models user-movie interactions as a bipartite graph, where users and movies are nodes connected by ratings. A random walk starts from a user or movie and moves through the graph, visiting connected nodes. The algorithm ranks movies based on how frequently they are visited during the walk. Random restarts to relevant nodes are introduced to keep the walk focused on relevant areas, improving personalization and depth of exploration.

---

4. Implementation Details

The implementation was done in Python using Pandas, NumPy, and Scikit-learn. The user-movie rating matrix was created using the `pivot()` function. For user-based and item-based filtering, missing values were filled with 0s or column-wise averages, depending on the approach.

For the graph-based recommender:

- An adjacency list was built to represent user-movie interactions.
- Each user was connected to all the movies they rated, and each movie was connected to the users who rated it.

- The random walk starts from either a user (for user-based walks) or a movie (for movie-based walks). At each step, the walker moves to a randomly selected neighbor node. A restart mechanism was implemented using highly rated movies or frequent raters, guiding the walker back to relevant parts of the graph.

- After the walk, the system tallies how often each movie was visited and ranks them to provide final recommendations.

---

5. Results and Evaluation

Example Outputs:

- **User-Based Collaborative Filtering**: For user 10, the top recommended movies included *In the Company of Men (1997)* and *Les Misérables (1995)*.

- **Item-Based Collaborative Filtering**: For the movie *Jurassic Park (1993)*, recommendations included *Top Gun (1986)* and *Empire Strikes Back (1980)*.

- **Pixie-Inspired Random Walk**: For user 1, movies like *Seven (Se7en)* and *Great Dictator, The (1940)* appeared among the top picks.

Comparison:

- The **collaborative filtering** methods gave consistent and interpretable results based on user history and similarity.

- The **Pixie-inspired algorithm** was better at exploring broader connections and uncovering hidden associations through the graph structure. It offered more diverse and sometimes surprising recommendations, which may be useful in discovery scenarios.

- The performance and quality of recommendations varied depending on parameters like walk length and restart probability. Longer walks captured deeper associations but could occasionally drift.

Limitations:

- Sparse data can reduce the effectiveness of user-based filtering.

- Graph-based recommendations depend on how well the graph captures meaningful user-item relationships.

- Genre or metadata-based filtering was not fully integrated due to missing genre information.

---

6. Conclusion

This project demonstrated how different recommendation algorithms can be used to generate personalized movie suggestions. While collaborative filtering approaches are straightforward and interpretable, graph-based algorithms provide a powerful and flexible alternative that captures deeper relationships.

Possible Improvements

While the current system performs reasonably well across all three recommendation strategies, there's definitely room to grow and improve its performance, especially when moving toward real-world usage.

1. Hybrid Recommendation Models
A great next step would be to combine user-based and item-based collaborative filtering into a hybrid system. This kind of setup balances the strengths of both methods—using patterns in user behavior while also accounting for how similar movies are to each other. Hybrid models also help reduce problems like "cold start" (when there's not enough data about a new user or movie) and tend to produce more stable and varied recommendations.

2. Use of Genre and Metadata
Adding extra context like genre, release year, or even directors and actors could make recommendations feel more personal. For instance, if someone tends to rate sci-fi movies highly, the system could prioritize similar content even if there aren't enough user ratings. This could be especially helpful for recommending newer or lesser-known movies that don't have a lot of data yet.

3. Content-Based Filtering
A content-based approach could be integrated as an enhancement. This method focuses more on the attributes of the movie itself rather than relying only on user preferences. It can help surface hidden gems by finding movies similar to what someone already likes—based on plot keywords, themes, or styles.

4. Graph Neural Networks (GNNs)
For a more advanced take, graph neural networks could be used. These models go beyond simple random walks by learning more complex relationships within the user-movie graph. They

can capture deeper, multi-step patterns that regular algorithms might miss, and they've shown strong performance in large-scale recommendation systems used by major platforms.

5. Matrix Factorization Techniques

Techniques like SVD (Singular Value Decomposition) or ALS (Alternating Least Squares) are also worth exploring. These methods work by uncovering hidden patterns in user-movie interactions, often leading to more accurate predictions, especially when dealing with sparse data.

6. Time & Behavior Awareness

Lastly, incorporating the time dimension—like how recently a movie was rated or watched—could boost relevancy. On top of that, using implicit feedback (clicks, watch time, skips) rather than just explicit ratings could give a fuller picture of what users enjoy, especially for passive viewers who don't leave many ratings.

With these additions, the system would be able to deliver smarter, more flexible recommendations that align better with real user behavior—something that's crucial if this were to be deployed on a larger platform.

The approaches developed here are directly applicable in real-world systems like Netflix, YouTube, and Spotify, where scalable, fast, and relevant recommendations significantly drive user engagement and satisfaction.